C# Ders - 5 Ekim 2026 Ders Notları|

1. Konsol Giriş / Çıkış (I/O) Komutları

- `Console.Write(...)`: Belirtilen değeri ekrana basar ve imleci **yazının bittiği yerde** bırakır. Bir sonraki çıktı aynı satırda devam eder.
- `Console.WriteLine(...)`: Belirtilen değeri ekrana basar ve otomatik olarak **bir alt satıra geçer**.
- `Console.ReadLine()`: Kullanıcı klavyeden metin girip **Enter** tuşuna basana kadar bekler; girilen tüm satırı `string` veri tipi olarak döner.
- `Console.Read()`: Kullanıcının klavyeden bastığı **ilk tek karakterin ASCII (sayısal) kodunu** `int` olarak döner.
- `Console.Clear()`: Konsol ekranındaki tüm yazıları temizler ve imleci sol üst köşeye (0, 0) taşır.

2. Kaçış Karakterleri (Escape Sequences)

Çift tırnak içinde özel görevleri tetiklemek veya derleyicinin hata vermesini önlemek için ters bölü (`\`) ile kullanılır:

- `\n` **(New Line)**: İmleci doğrudan bir alt satıra indirir.
- `\t` **(Horizontal Tab)**: Metinler arasına sekme (genellikle 4 veya 8 boşluk) boyutu kadar boşluk bırakır, sütun hizalamada kullanılır.
- `\r` **(Carriage Return)**: Satır başına döner. Kodda geçen `\rPazartesi` ifadesi, aynı satırın başına dönüp oradaki karakterlerin üzerine *"Pazartesi "* yazar.
- `\\`: Ters bölü işaretinin kendisini ekrana yazdırmak için kullanılır (çünkü tek başına `\` kaçış karakteri başlatır).
- `\"`: Metin içinde çift tırnak işaretini string sınırını bozmadan yazdırmak için kullanılır.

3. Değişken Tanımlama ve İlkel Veri Tipleri (Data Types)

C#'ta değişken tanımlama sözdizimi: `VeriTipi DeğişkenAdı = Değer;` şeklindedir.

Tam Sayı Tipleri (Tamsayılar)

Bellekte kapladıkları byte ve işaretli/işaretsiz olmalarına göre ayrılırlar (`u` ön eki *unsigned / işaretsiz* anlamına gelir, yalnızca sıfır ve pozitif değerleri tutar):

Tip	Boyut	Aralık	Not
`byte`	1 Byte	`0` ile `255`	İşaretsiz tam sayı
`sbyte`	1 Byte	`-128` ile `127`	İşaretli tam sayı
`short`	2 Byte	`-32.768` ile `32.767`	İşaretli tam sayı
`ushort`	2 Byte	`0` ile `65.535`	İşaretsiz tam sayı
`int`	4 Byte	`~ -2.14 milyar` ile `~ +2.14 milyar`	En yaygın kullanılan varsayılan tam sayı tipi
`uint`	4 Byte	`0` ile `~ 4.29 milyar`	İşaretsiz tam sayı
`long`	8 Byte	`~ -9 x 10¹⁸` ile `~ +9 x 10¹⁸`	Çok büyük tam sayılar için

Ondalıklı Sayı Tipleri

Hassasiyet (virgülden sonraki basamak doğruluğu) ve performans farkları bulunur:

- `float`: 4 Byte yer kaplar. 6–9 basamak hassasiyeti vardır. Değer sonuna zorunlu olarak `f` veya `F` eki alır (`3.14f`).
- `double`: 8 Byte yer kaplar. 15–17 basamak hassasiyeti vardır. C#'ın varsayılan ondalıklı tipidir.
- `decimal`: 16 Byte yer kaplar. 28–29 basamak hassasiyeti vardır. Finansal, para ve muhasebe hesaplamalarında yuvarlama hatalarını önlemek için kullanılır. Değer sonuna `m` veya `M` eki alır.

Metinsel ve Mantıksal Tipler

- `char`: 2 Byte yer kaplar. Tek tırnak (`'A'`) içinde yalnızca tek bir Unicode karakter tutar.

- `string`: Çift tırnak (`"..."`) içinde sınırsız metin blokları tutan referans tipidir.

4. Metin Birleştirme ve Formatlama Yöntemleri

Kodda aynı çıktıyı elde etmek için üç farklı yaklaşım kullanılmıştır:

1. **Artı (`+`) Operatörü ile Birleştirme:**C#

   ```
   "Ad :" + name + " " + name2 + " Soyad: " + lastname;
   ```

   Klasik yöntemdir; değişkenler metne çevrilip ardı ardına eklenir.

2. **Index / Composite Formatting (`string.Format` Stili):**C#

   ```
   Console.WriteLine("Ad: {0} {4}, Soyad: {1}", name, lastname, city, 45, name2);
   ```

   Metin içine süslü parantez içinde indisler (`{0}`, `{1}`, `{4}`) açılır, virgülden sonra verilen parametreler sırasına göre bu indislerin yerine yerleştirilir.

3. **String Interpolation (`$` İşareti):**C#

   ```
   Console.WriteLine($"Ad: {name} {name2},\nSoyad: {lastname}");
   ```

   Metin başına `$` konur; değişkenler ve matematiksel işlemler doğrudan süslü parantez içine (`{...}`) yazılır. En okunaklı ve modern yaklaşımdır.

5. Standart Sayısal Format Belirleyiciler (Format Specifiers)

`{değer:format}` sözdizimiyle sayıların ekrandaki görünümü biçimlendirilir:

- `:c` / `:c2` **(Currency - Para Birimi):** Sistem para birimini (₺, $, €) ve kuruş basamağını ekler. `c2` virgülden sonra iki basamak gösterir.
- `:n` / `:n2` **(Number - Sayı):** Sayıyı binlik basamaklara ayırır (nokta/virgül ayrımıyla) ve belirtilen basamak kadar ondalık bırakır (örn: `70.000,35`).

- `:f` / `:F2` **(Fixed-Point - Sabit Noktalı):** Sayıyı binlik ayracı olmadan, doğrudan sabit basamaklı ondalık formatında gösterir (örn: `70000.35`).
- `:D8` **(Decimal Leading Zeros):** Sadece tam sayılarda çalışır; sayının toplam basamak sayısı 8 olana kadar başına sıfır (`0`) ekler (örn: `78965` → `00078965`).
- `:x` **(Hexadecimal - Onaltılık):** Sayıyı 16'lık tabandaki karşılığına dönüştürür.
- `:e` **(Exponential - Bilimsel Gösterim):** Sayıyı üstel formatta (`7.896579e+011`) ifade eder.
- `:p` **(Percentage - Yüzde):** Sayıyı 100 ile çarpar ve yanına `%` işareti ekler (örn: `0.45` → `%45,00`).

6. Konsol Ortam ve Görsel Ayarları

- `Console.Title`: Konsol penceresinin en üst başlık çubuğunda yazacak metni belirler.
- `Console.ForegroundColor`: Konsola yazdırılacak metinlerin yazı rengini ayarlar (`ConsoleColor.Green` vb.).
- `Console.BackgroundColor`: Konsolun arka plan rengini ayarlar (`ConsoleColor.Magenta` vb.).
- `Console.OutputEncoding = System.Text.Encoding.UTF8;`: Konsolun Türkçe karakterleri (ş, ğ, ı, İ, ö, ç) ve özel Unicode sembolleri bozulmadan (`?` çıkmadan) doğru görüntülemesini sağlar.